université
PARIS-SACLAY

# RAPPORT DE TP

*Programmation des architectures SoC hétérogènes CPU-FPGA*


*Auteur :*

Gaëtan BOUCHY
Thomas ABOU JAOUDE

*Enseignant :*

M. EL OUARDI


Nous attestons que ce travail est original, qu'il est le fruit d'un travail commun en binôme et qu'il a été rédigé de manière autonome.

À Orsay, le 21/01/2022

université
PARIS-SACLAY

# Sommaire

Sommaire ..... 2
1. Analyse des performances de l'algorithme de Sobel ..... 3
1.1 Calcul de la complexité par pixel ..... 3
2. Prise en main de la carte et de ses outils ..... 5
2.1 Analyse du code de la machine "host" (CPU ARM) ..... 5
2.1.1 Analyser le fichier Makefile ..... 5
2.1.2 Quelle est la fonctionnalité du programme principal ? ..... 5
2.1.3 Pensez-vous que la fonction exécutable sur le processeur ARM est optimisée ? .. 5
2.1.4 Comment se fait la mesure du temps d'exécution ? ..... 6
2.1.5 Comment sont transférées les données au FPGA ? ..... 6
2.2 Analyse du code de la machine "Device" (FPGA) ..... 7
2.2.1 Pensez-vous que la fonction implémentée sur le FPGA est parallélisée ? ..... 7
2.2.2 Si oui, quelle serait l'architecture synthétisée sur l'accélérateur FPGA ? ..... 8
3. Évaluation des performances ..... 9
3.1 Comparaison des temps d'exécution et des temps de latence ..... 9
3.2 Calcul de la complexité par pixel ..... 13
3.3 Analyse des images de sortie ..... 16
4. Évaluation des performances avec un autre algorithme de complexité différente ..... 18
4.1 Explication et choix de l'algorithme ..... 18
4.2 Implémentation de l'algorithme (langage OpenCL) ..... 19
4.3 Implémentation de l'algorithme (langage C++) ..... 21
4.4 Différence de complexité des algorithmes ..... 22
4.4.1 Algorithme de Sobel ..... 22
4.4.2 Algorithme de flou Gaussien ..... 22

université PARIS-SACLAY

# 1. Analyse des performances de l'algorithme de Sobel

## 1.1 Calcul de la complexité par pixel

La complexité par pixel ou « Cpp » mesure la complexité d'un algorithme de traitement d'image en fonction du nombre de pixels (résolution) de l'image et du temps de cycle nécessaire pour traiter chaque pixel. Plus la Cpp est faible, plus l'algorithme est efficace. La complexité par pixel peut être obtenu via la formule suivante : $Cpp = \frac{Temps\ de\ traitement}{résolution * temps\ de\ cycle}$

Nous allons maintenant calculer la Cpp pour un algorithme de Sobel qui est un algorithme de détection de bords utilisé pour extraire les contours d'une image. Il est basé sur le filtre de Sobel qui est utilisé pour calculer les éléments de gradient de l'image. Cela est utile pour l'analyse d'image, la reconnaissance de formes et les systèmes de vision par ordinateur.

Nous allons maintenant calculer plusieurs Cpp pour différentes résolutions d'images. Dans notre formule, le temps de cycle se calcule ainsi : $Temps\ de\ cycle = \frac{1}{Fréquence\ du\ CPU}$

Nous avons calculé la Cpp de l'algorithme de Sobel sur un ordinateur avec processeur Intel d'une fréquence de 2.3 GHz soit 2 308 MHz. Ainsi le temps de cycle vaut :

$$Temps\ de\ cycle = \frac{1}{Fréquence\ du\ CPU} = \frac{1}{2308 * 10^6} = 4.34 * 10^{-10}$$

| Temps de traitement (en seconde) | Résolution (en pixel) | Temps de cycle (en seconde) | Cpp |
|---|---|---|---|
| $15*10^{-6}$ | 60 | $4.34*10^{-10}$ | 575,99 |
| $868*10^{-6}$ | 5 600 | $4.34*10^{-10}$ | 357,11 |
| $6\ 536*10^{-6}$ | 68 600 | $4.34*10^{-10}$ | 219,51 |
| $46\ 830*10^{-6}$ | 331 776 | $4.34*10^{-10}$ | 325,20 |
| $65\ 380*10^{-6}$ | 563 000 | $4.34*10^{-10}$ | 267,55 |
| $89\ 550*10^{-6}$ | 1 350 000 | $4.34*10^{-10}$ | 152,83 |
| $1\ 003\ 712*10^{-6}$ | 14 000 000 | $4.34*10^{-10}$ | 165,18 |
| $3\ 741\ 165*10^{-6}$ | 56 000 000 | $4.34*10^{-10}$ | 153,92 |

*Tableau 1 : Calcul du Cpp pour différentes résolutions d'images*

Comme nous pouvons le voir, il semblerait que plus la résolution est grande, plus le temps de traitement l'est aussi. Cela s'explique par le fait que si l'on a plus de pixels dans une image, il y a plus d'informations à traiter. Si l'algorithme de traitement utilisé est efficace, il devrait être capable de traiter ces informations supplémentaires en utilisant le même temps de traitement ou même moins de temps que pour une image de moins bonne résolution, ce qui entraîne une réduction de la complexité par pixel pour des images de plus grandes tailles.

Si nous essayons de tracer le graphique de la Cpp en fonction de la résolution, voici ce que nous obtenons (le point avec comme résolution 56 000 000 a été omis) :

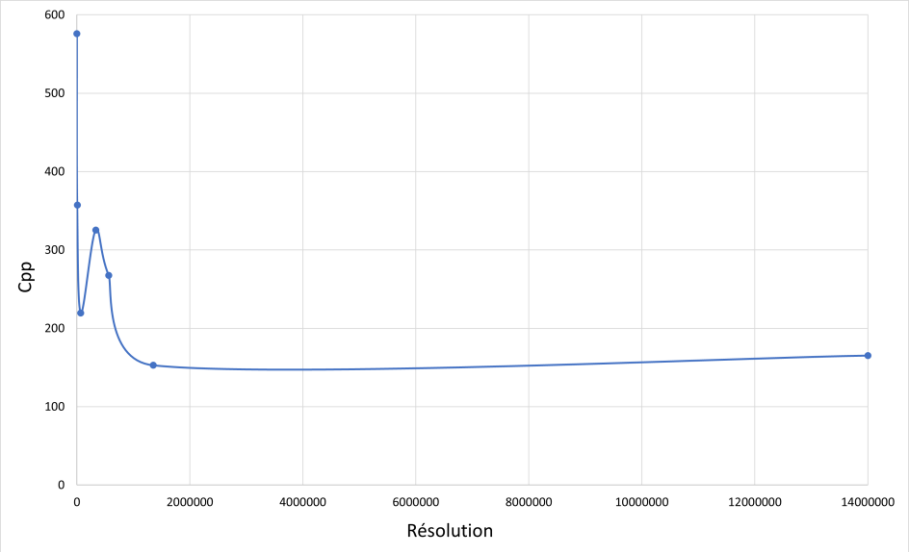



*Figure 1 : Courbe de la Cpp en ordonnée et de la résolution en abscisse*

Comme nous pouvons le voir, il semblerait que notre courbe oscille pour des valeurs de résolution assez faible. Cela pourrait s'expliquer par deux phénomènes :

- La qualité de l'image : Les images de faible résolution peuvent être plus difficiles à traiter en raison de la perte d'informations. Les détails de l'image peuvent être flous ce qui rend plus difficile la détection des bords et l'extraction des contours
- Le bruit dans l'image : Les images avec un bruit élevé peuvent entraîner une augmentation de la complexité de l'algorithme, car il est nécessaire de traiter des informations superflues

# 2. Prise en main de la carte et de ses outils

## 2.1 Analyse du code de la machine "host" (CPU ARM)

### 2.1.1 Analyser le fichier Makefile

Un fichier Makefile est un fichier utilisé par la commande "make" pour automatiser la compilation et l'installation de programmes. La commande "make" utilise les instructions contenues dans le fichier Makefile pour déterminer comment compiler et lier les différents fichiers de code source en un programme exécutable.

Ce fichier Makefile est utilisé pour automatiser les tâches de compilation et de génération d'exécutables. Ce fichier définit des cibles, des dépendances et des commandes pour construire l'exécutable « host » dans le répertoire « bin ». Il utilise un compilateur C++ (« arm-linux-gnueabihf-g++ » dans le fichier) pour construire l'exécutable.

Le Makefile comporte également des options de compilation telles que « -g » pour activer le débogage ou « -O2 » pour l'optimisation. Il utilise également des bibliothèques externes comme OpenCV, des variables d'environnement telles que INTELFPGAOCLSDKROOT pour pointer vers le répertoire d'installation de l'Intel(R) FPGA SDK for OpenCL(TM) et des commandes de système pour configurer les options de compilation et de liaison pour OpenCL.

### 2.1.2 Quelle est la fonctionnalité du programme principal ?

Le programme principal peut se définir comme étant « display.cpp » dans le fichier « Host ». La fonction principale du programme est d'appliquer un filtre de Sobel à une image en utilisant OpenCV pour charger et afficher l'image, et OpenCL pour accélérer le traitement de l'image en utilisant le processeur graphique. Il y a également des instructions pour la capture des temps d'exécution. Un filtre de Sobel est appliqué en utilisant des matrices de filtrage et des boucles pour parcourir les pixels de l'image. Il y a aussi des fonctions pour initialiser OpenCL et libérer les ressources utilisées.

### 2.1.3 Pensez-vous que la fonction exécutable sur le processeur ARM est optimisée ?

Cette fonction de filtrage de Sobel pour le processeur ARM se base sur deux boucles imbriquées qui parcourent tous les pixels de l'image d'entrée. Pour chaque pixel, elle calcule les valeurs de gradient en utilisant les matrices de filtrage « Gx » et « Gy ». Ces valeurs sont ensuite utilisées pour déterminer si chaque pixel est un bord ou non en utilisant une seuil de comparaison. Cette fonction n'est pas très optimisée pour deux raisons :

- Tout d'abord l'utilisation de boucles imbriquées pour parcourir tous les pixels de l'image ce qui peut entraîner une faible performance pour les images de grandes tailles
- Utilisation de variables entières pour stocker les valeurs de pixel, ce qui peut causer des dépassements de mémoire pour les images de grandes tailles

Il est donc possible d'optimiser cette fonction en utilisant des techniques de parallélisation pour parcourir les pixels et en utilisant des types de données plus efficaces pour stocker les valeurs de pixel.

### 2.1.4 Comment se fait la mesure du temps d'exécution ?

Dans ce code, le temps d'exécution est mesuré en grâce à la fonction « gettimeofday » de la bibliothèque « sys/time.h ». La fonction « gettimeofday » prend en entrée deux paramètres : un pointeur vers un objet de type « timeval » qui contiendra l'heure (ici « begin_l » pour le début de l'exécution et « end_l » pour la fin de l'exécution) et un pointeur vers un objet de type "timezone" qui n'est pas utilisé ici.

La différence entre les valeurs contenues dans ces objets « timeval » est ensuite calculée pour donner la durée de l'exécution en microsecondes qui est ensuite affichée via les printfs.

### 2.1.5 Comment sont transférées les données au FPGA ?

Dans ce code, les données sont transférées du CPU vers le FPGA en utilisant la fonction OpenCL « clEnqueueWriteBuffer ». Cette fonction prend en entrée un pointeur de file d'attente, un tampon de mémoire (in_buffer), un indicateur indiquant si les données doivent être copiées immédiatement ou non (CL_FALSE), la taille des données à transférer (sizeof(unsigned int) * IMG_ROWS * IMG_COLS) et le pointeur vers les données à transférer (input).

Avant de transférer les données, un timestamp est pris pour mesurer le temps d'exécution. Il est ensuite utilisé pour calculer le temps écoulé entre le début et la fin de la fonction, qui est ensuite affiché avec le message « data transfer CPU to FPGA ».

Après l'appel de la fonction « clEnqueueWriteBuffer », la fonction « clFinish » est utilisée pour s'assurer que toutes les commandes ont été exécutées avant de poursuivre.

## 2.2 Analyse du code de la machine "Device" (FPGA)

### 2.2.1 Pensez-vous que la fonction implémentée sur le FPGA est parallélisée ?

La fonction implémentée sur le FPGA est parallélisée car elle utilise des instructions OpenCL pour exécuter un filtre Sobel sur une image. Nous remarquons deux points qui nous font fortement penser que cette fonction est parallèle et qui sont :
- L'utilisation du mot « __**kernel** » qui est une annotation OpenCL qui indique qu'une fonction est un « noyau OpenCL ». Ce sont des fonctions qui tournent sur des FPGAs ou des GPUs. Elles peuvent être exécutées en parallèle sur plusieurs unités de calcul, chacun utilisant des données différentes pour augmenter les performances
- Les « **pragma unroll** » qui sont des directives de pré-processeur utilisées pour demander à un compilateur de dérouler une boucle. Cela permet à la fonction de traiter plusieurs pixels à chaque itération de boucle, ce qui augmente la parallélisation

En résumé, ce code utilise des techniques de parallélisation pour accélérer le traitement de l'image sur FPGA.

### 2.2.2 Si oui, quelle serait l'architecture synthétisée sur l'accélérateur FPGA ?

L'architecture synthétisée est la traduction physique du code pour un FPGA. Cela décrit comment le code sera implémenté sur les différents éléments du FPGA. Cela inclut les choix de placement, de routage, d'optimisation de la vitesse et de la consommation d'énergie. L'architecture synthétisée est générée par un outil de synthèse de RTL qui prend en entrée un description de haut niveau tel que le VHDL, l'OpenCL, le C/C++ et génère un fichier de description de bas niveau qui décrit l'architecture physique de la matrice de logiciels programmables sur laquelle le code sera exécuté.

C'est pourquoi nous avons utilisé la commande « **aoc device/fichier.cl --sw-dimm-partition -o bin/fichier.aocx--report** ». La commande aoc (Altera OpenCL Compiler) est utilisée pour compiler des fichiers OpenCL pour être exécutés sur des accélérateurs FPGA Altera. Elle prend en entrée un fichier OpenCL et génère un fichier binaire qui peut être chargé sur un FPGA pour exécuter le code OpenCL.

L'option « **--report** » permet de générer un rapport détaillant les informations sur la synthèse, l'utilisation des ressources, et les performances que nous retrouvons en effet dans le fichier.aocx.

Si nous nous penchons sur le fichier sobel.aocx :

```
ALUTs: 7961
Registers: 8,744
Logic utilization: 4,953 / 32,070 ( 15 % ) ( 15 % )
I/O pins: 103 / 457 ( 23 % )
DSP blocks: 4 / 87 ( 5 % )
Memory bits: 201,104 / 4,065,280 ( 5 % )
M10K blocks: 50 / 397 ( 13 % )
Actual clock freq: 148.379999896
Kernel fmax: 148.38
1x clock fmax: 148.38
2x clock fmax: 10000
Highest non-global fanout: 2548
```

*Figure 2 : Fichier Sobel.aocx*

Nous voyons en effet à la fin du fichier des informations sur l'utilisation des ressources du FPGA : 7 961 unités arithmétique et logique seront utilisées, 8 744 registres, 4 blocks DSP, 201 104 bits mémoires, …

# 3. Évaluation des performances

## 3.1 Comparaison des temps d'exécution et des temps de latence

Après avoir analysé le fonctionnement du code sur CPU et sur FPGA, nous allons maintenant évaluer les performances en comparant les temps d'exécution de l'algorithme de Sobel sur ces deux supports pour différentes taille d'image :

| Résolution (en pixel) | Temps d'exécution sur FPGA (en seconde) | Temps d'exécution sur CPU (en seconde) |
|---|---|---|
| 5 600 (56x100) | $341*10^{-6}$ | $1\ 625*10^{-6}$ |
| 202 800 (338x600) | $1\ 676*10^{-6}$ | $62\ 074*10^{-6}$ |
| 529 620 (546x970) | $3\ 894*10^{-6}$ | $158\ 071*10^{-6}$ |
| 680 900 (619x1100) | $4\ 965*10^{-6}$ | $212\ 759*10^{-6}$ |
| 1 440 000 (900x1600) | $9\ 540*10^{-6}$ | $445\ 594*10^{-6}$ |
| 2 480 100 (1181x2100) | $16\ 037*10^{-6}$ | $761\ 401*10^{-6}$ |
| 3 801 200 (1462x2600) | $25\ 103*10^{-6}$ | $1\ 206\ 229*10^{-6}$ |
| 5 403 300 (1743x3100) | $36\ 518*10^{-6}$ | $1\ 725\ 795*10^{-6}$ |
| 7 286 400 (2024x3600) | $48\ 528*10^{-6}$ | $2\ 297\ 232*10^{-6}$ |
| 9 450 500 (2305x4100) | $65\ 359*10^{-6}$ | $3\ 055\ 996*10^{-6}$ |
| 11 895 600 (2586x4600) | $79\ 349*10^{-6}$ | $3\ 813\ 500*10^{-6}$ |
| 14 621 700 (2867x5100) | $97\ 926*10^{-6}$ | $4\ 526\ 759*10^{-6}$ |

*Tableau 2 : Tableau de comparaison du temps d'exécution de l'algorithme de Sobel sur CPU et FPGA en fonction de la résolution d'une image*

Si nous récupérons les données du tableau et que nous les représentons graphiquement, voici ce que nous obtenons :

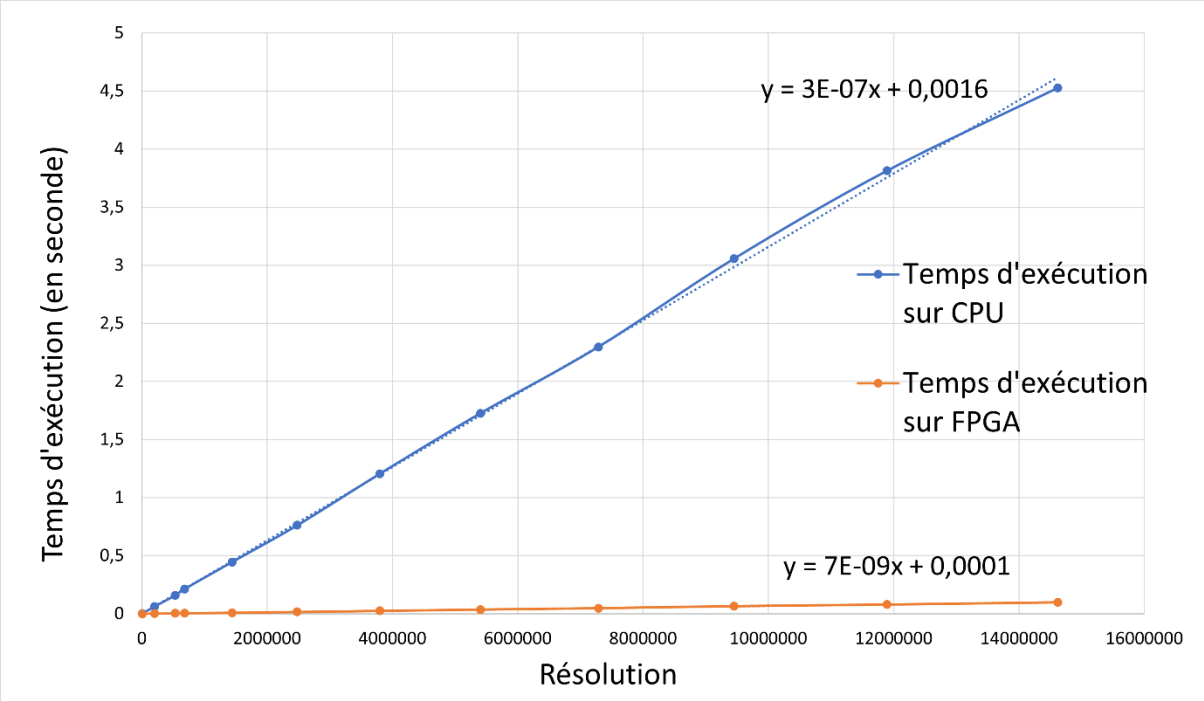



*Figure 3 : Courbe du temps d'exécution de l'algorithme de Sobel sur CPU et FPGA en fonction de la résolution d'une image*

L'analyse des données montre que l'algorithme de Sobel est effectué beaucoup plus rapidement sur FPGA que sur CPU. En comparant les temps d'exécution, on constate que les temps d'exécution sur FPGA sont en moyenne plusieurs dizaines de fois plus rapides que sur CPU ($Moyenne = 3 * 10^{-7}/7 * 10^{-9} = 42.85$, moyenne des rapports des différents temps d'exécution du CPU sur ceux du FPGA d'après les équations linéaires fournies présentes sur le graphe). Cela montre que l'utilisation d'un FPGA permet de réaliser l'algorithme de Sobel beaucoup plus rapidement qu'un CPU.

Cette différence de performance est due à la nature différente des architectures des FPGA et des CPU. Les FPGA sont conçus pour l'exécution de calculs parallèles, ce qui leur permet de traiter de nombreuses tâches simultanément. En revanche, les CPU sont conçus pour l'exécution séquentielle de tâches, ce qui les rend plus adaptés aux tâches de calculs séquentielles. Dans le cas de l'algorithme de Sobel, qui nécessite un traitement parallèle des pixels de l'image, l'utilisation d'un FPGA permet des performances beaucoup plus élevées qu'un CPU.

Cependant, ce que nous avons omis lors du graphique précédent sont les temps de transfert des données du côté du FPGA. En effet, au temps d'exécution de l'algorithme de Sobel sur FPGA s'ajoute également les temps de transfert des données du CPU au FPGA puis du FPGA au CPU une fois que l'algorithme a été effectué sur la cible :

| Résolution (en pixel) | Temps d'exécution sur FPGA (en seconde) | Temps d'exécution sur CPU (en seconde) | Temps de transfert des données du CPU au FPGA (en seconde) | Temps de transfert des données du FPGA au CPU (en seconde) |
|---|---|---|---|---|
| 5 600 | $341*10^{-6}$ | $1\ 625*10^{-6}$ | $450*10^{-6}$ | $235*10^{-6}$ |
| 202 800 | $1\ 676*10^{-6}$ | $62\ 074*10^{-6}$ | $7\ 682*10^{-6}$ | $6\ 233*10^{-6}$ |
| 529 620 | $3\ 894*10^{-6}$ | $158\ 071*10^{-6}$ | $18\ 957*10^{-6}$ | $15\ 765*10^{-6}$ |
| 680 900 | $4\ 965*10^{-6}$ | $212\ 759*10^{-6}$ | $24\ 399*10^{-6}$ | $21\ 005*10^{-6}$ |
| 1 440 000 | $9\ 540*10^{-6}$ | $445\ 594*10^{-6}$ | $52\ 851*10^{-6}$ | $44\ 644*10^{-6}$ |
| 2 480 100 | $16\ 037*10^{-6}$ | $761\ 401*10^{-6}$ | $88\ 544*10^{-6}$ | $74\ 757*10^{-6}$ |
| 3 801 200 | $25\ 103*10^{-6}$ | $1\ 206\ 229*10^{-6}$ | $134\ 168*10^{-6}$ | $117\ 722*10^{-6}$ |
| 5 403 300 | $36\ 518*10^{-6}$ | $1\ 725\ 795*10^{-6}$ | $199\ 395*10^{-6}$ | $167\ 249*10^{-6}$ |
| 7 286 400 | $48\ 528*10^{-6}$ | $2\ 297\ 232*10^{-6}$ | $268\ 622*10^{-6}$ | $225\ 760*10^{-6}$ |
| 9 450 500 | $65\ 359*10^{-6}$ | $3\ 055\ 996*10^{-6}$ | $348\ 198*10^{-6}$ | $292\ 840*10^{-6}$ |
| 11 895 600 | $79\ 349*10^{-6}$ | $3\ 813\ 500*10^{-6}$ | $438\ 041*10^{-6}$ | $368\ 230*10^{-6}$ |
| 14 621 700 | $97\ 926*10^{-6}$ | $4\ 526\ 759*10^{-6}$ | $516\ 879*10^{-6}$ | $435\ 889*10^{-6}$ |

*Tableau 3 : Tableau de comparaison du temps d'exécution de l'algorithme de Sobel sur CPU et FPGA en fonction de la résolution d'une image*

Avec ces nouvelles informations nous pouvons calculer le temps de latence qui est le temps total nécessaire pour transférer les données et les traiter. Voici donc la formule du temps de latence pour notre FPGA :

$$\text{Temps de latence du FPGA} = \text{Temps de transfert des données du CPU au FPGA} + \text{temps d'exécution sur FPGA} + \text{Temps de transfert des données du FPGA au CPU}$$

En calculant ainsi le temps de latence du FPGA, voici le nouveau graphique que nous obtenons :

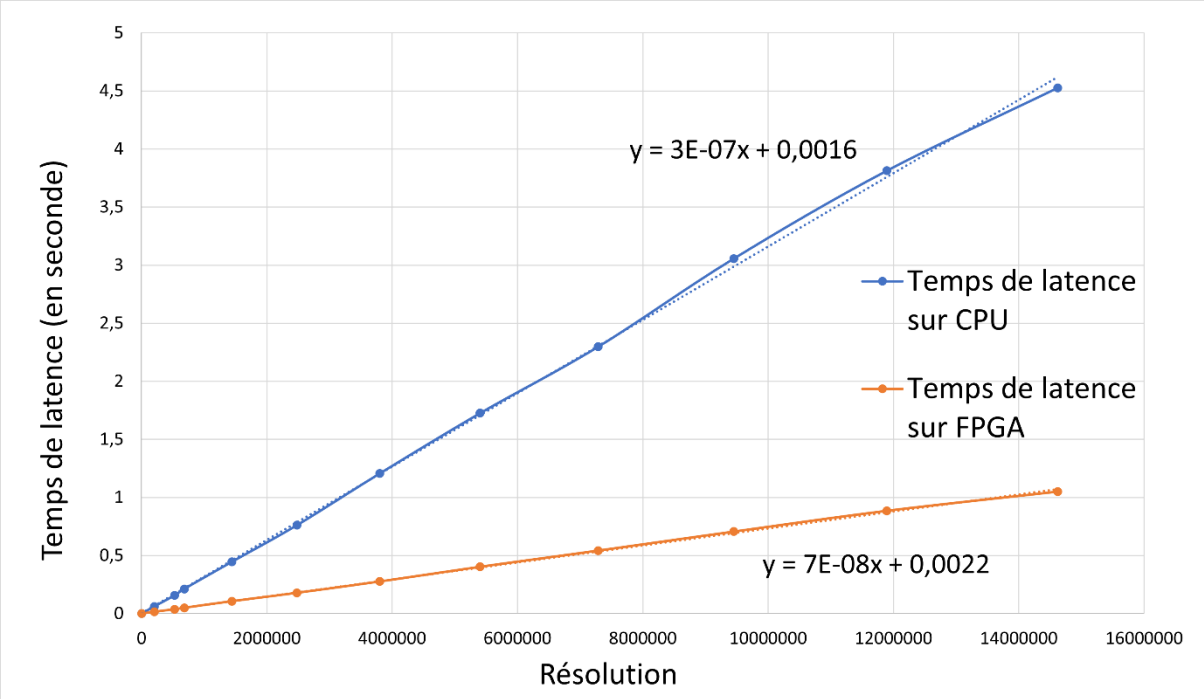



*Figure 4 : Courbe du temps de latence de l'algorithme de Sobel sur CPU et FPGA en fonction de la résolution d'une image*

Comme nous pouvons le voir, le temps de latence du FPGA reste bien inférieur à celui du CPU mais l'écart s'est nettement réduit. En effet, nous avions en moyenne des temps d'exécutions sur CPU 42.85 fois supérieur à ceux sur FPGA. Actuellement, avec ce nouveau modèle, nous avons en moyenne des temps de latence sur CPU 4.285 fois supérieur à ceux sur FPGA ($Moyenne = 3 * 10^{-7}/7 * 10^{-8} = 4.285$, d'après les équations linéaires fournies présentes sur le graphe).

## 3.2 Calcul de la complexité par pixel

La complexité par pixel ou « Cpp » mesure la complexité d'un algorithme de traitement d'image en fonction du nombre de pixels (résolution) de l'image et du temps de cycle nécessaire pour traiter chaque pixel. Plus la Cpp est faible, plus l'algorithme est efficace. La complexité par pixel peut être obtenu via la formule suivante : $Cpp = \frac{\text{Temps de traitement}}{\text{résolution} * \text{temps de cycle}}$

Nous allons maintenant calculer plusieurs Cpp pour différentes résolutions d'images. Dans notre formule, le temps de cycle se calcule ainsi : $Temps\ de\ cycle = \frac{1}{Fréquence\ du\ CPU}$

Nous avons calculé la Cpp de l'algorithme de Sobel sur un processeur ARM Cortex-A9 à 800 MHz et sur un FPGA Cyclone V à 50 MHz. Ainsi le temps de cycle vaut :

$$Temps\ de\ cycle\ ARM\ Cortex-A9 = \frac{1}{Fréquence\ du\ CPU} = \frac{1}{800 * 10^6} = 1.25 * 10^{-9}$$

$$Temps\ de\ cycle\ FPGA\ Cyclone\ V = \frac{1}{Fréquence\ du\ FPGA} = \frac{1}{50 * 10^6} = 2 * 10^{-8}$$

En récupérant les temps de latence pour les différentes résolutions, nous pouvons alors facilement calculer les Cpp pour tous les cas :

| Résolution (en pixel) | Cpp CPU ARM Cortex-A9 | Cpp FPGA Cyclone V |
|---|---|---|
| 5 600 | 232,14 | 9,16 |
| 202 800 | 244,86 | 3,84 |
| 529 620 | 238,76 | 3,64 |
| 680 900 | 249,97 | 3,69 |
| 1 440 000 | 247,55 | 3,71 |
| 2 480 100 | 245,60 | 3,61 |
| 3 801 200 | 253,86 | 3,64 |
| 5 403 300 | 255,51 | 3,73 |
| 7 286 400 | 252,22 | 3,72 |
| 9 450 500 | 258,69 | 3,73 |
| 11 895 600 | 256,46 | 3,72 |
| 14 621 700 | 247,67 | 3,59 |

*Tableau 4 : Calcul du Cpp sur CPU et FPGA pour différentes résolutions d'images*

Les résultats montrent que la Cpp pour le CPU est nettement plus élevé que celui du FPGA pour toutes les résolutions de pixel testées. Cela signifie que pour un même nombre de pixels, le CPU prend plus de temps pour traiter les images que le FPGA.

De plus, en observant les résultats pour le FPGA, on constate que la Cpp reste relativement stable pour les différentes résolutions de pixels. Cela indique que le FPGA a une performance globalement constante. Il en est à peu près de même pour le CPU.

En résumé, les résultats montrent que le FPGA Cyclone V est plus performant que le CPU ARM Cortex-A9 pour le traitement d'images en termes de temps de traitement par pixel comme le montre le graphique :

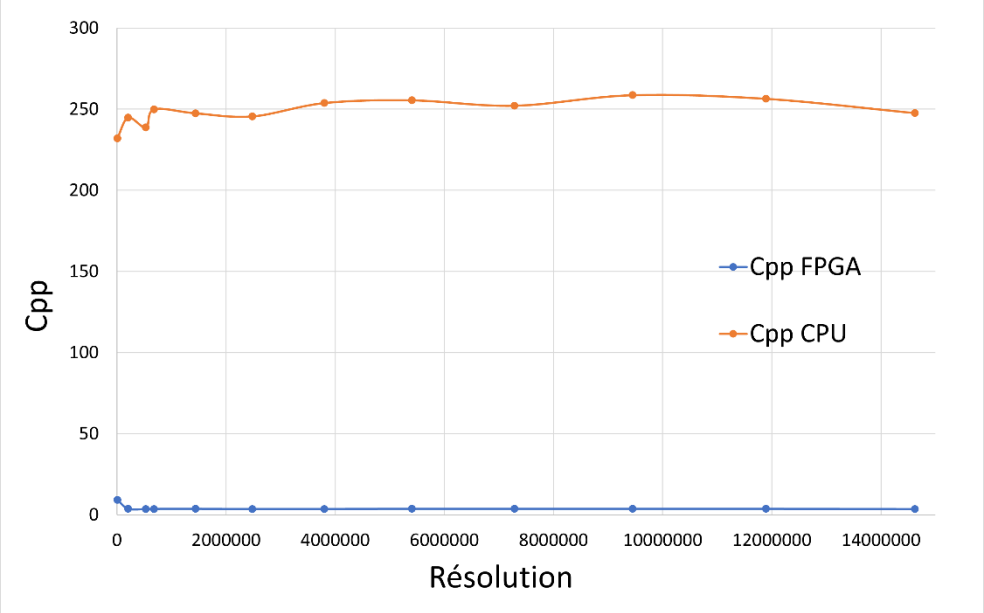



*Figure 5 : Courbe de la Cpp en ordonnée et de la résolution en abscisse pour un CPU et un FPGA réalisant le même algorithme de Sobel*

Concernant le graphique de la complexité par pixel du FPGA uniquement, voici ce que nous obtenons :

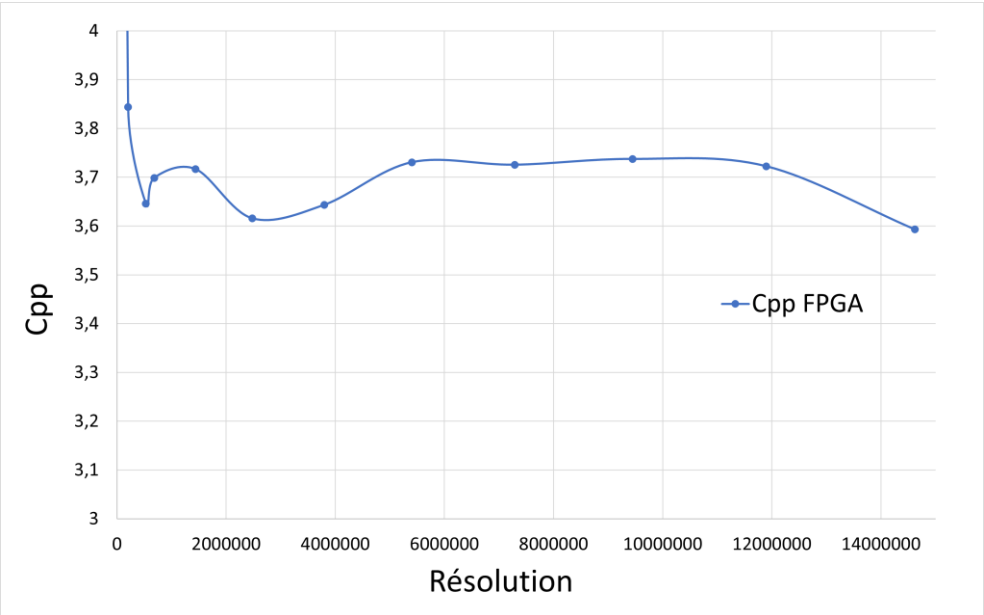



*Figure 6 : Courbe de la Cpp en ordonnée et de la résolution en abscisse pour un FPGA*

Concernant le graphique de la complexité par pixel du CPU uniquement, voici ce que nous obtenons :

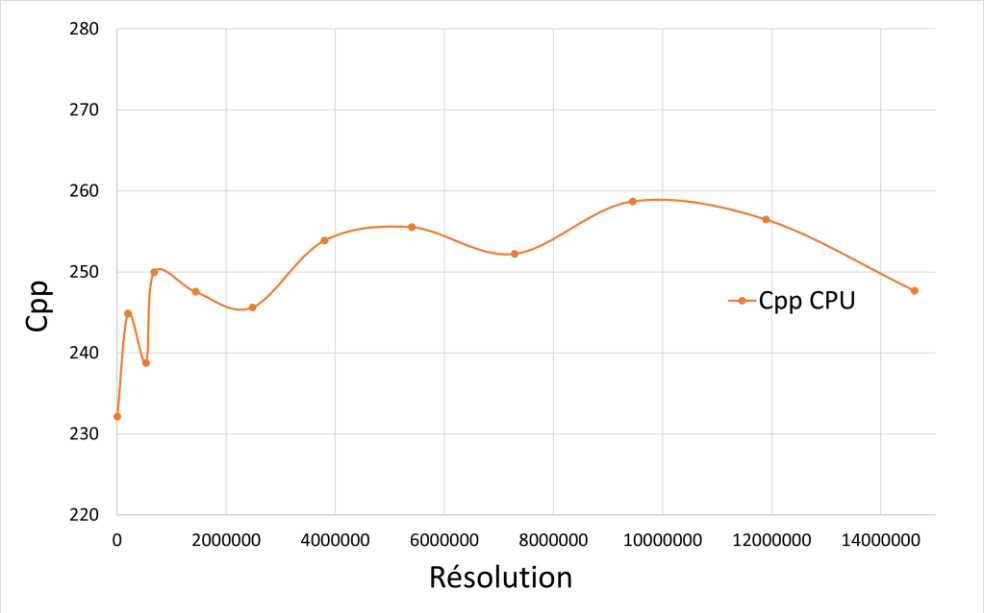



*Figure 7 : Courbe de la Cpp en ordonnée et de la résolution en abscisse pour un CPU*

On remarque globalement que les performances sont constantes malgré les quelques variations pour les différentes résolutions d'images. Le seul point qui change est celui pour une très faible résolution ou le CPU obtient un meilleur résultat comparé à ces autres résolutions.

Quant au FPGA, il obtient un mauvais résultat. Cela est lié au fait que le temps de transfert des données joue un important rôle vis-à-vis du temps d'exécution de l'algorithme.

## 3.3 Analyse des images de sortie

Voici l'image d'origine avec comme dimensions 970 pixel de largeur et 546 de hauteur (soit une résolution de 529 620) sur laquelle nous avons appliqué un algorithme de Sobel :



*Figure 8 : Image d'origine avant application d'un filtre de Sobel*

Voici les images après application d'un filtre de Sobel (sur CPU et sur FPGA) sur l'image d'orgine possédant les mêmes dimensions (970x546):

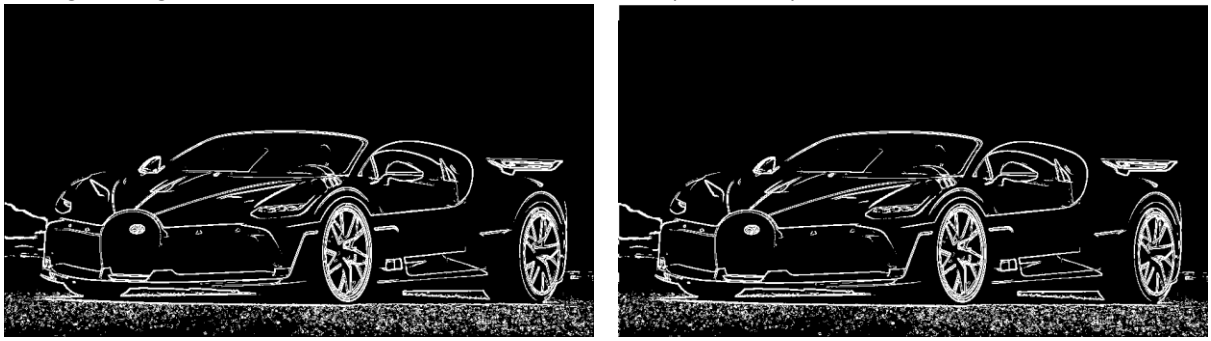


*Figure 9 : Image après application d'un filtre de Sobel (sur CPU à gauche et sur FPGA à droite)*

Comme nous pouvons le voir, les résultats sont bons, nous avons beaucoup de détails sur la carrosserie de la voiture, les pneus avec les jantes ainsi que certains aspects concernant le sol.

Nous avons ensuite redimensionné l'image d'origine en une image aillant comme dimensions 3 600x2 024 pixels et voici ce que nous obtenons en sortie après l'application d'un filtre de Sobel :

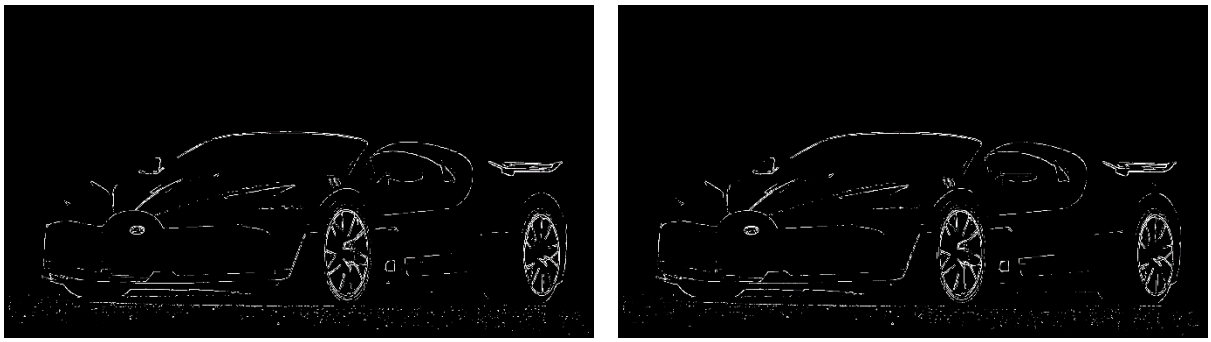


*Figure 10 : Image après application d'un filtre de Sobel (sur CPU à gauche et sur FPGA à droite)*

Nous remarquons que l'application d'un filtre de Sobel sur une image de plus grande résolution engendre des résultats beaucoup moins bons.

Cela est dû au fait que l'algorithme de Sobel utilise un filtre de convolution qui parcourt chaque pixel de l'image pour détecter les contours. Il se base sur la différence de luminosité entre chaque pixel et ses voisins pour détecter les contours. Plus l'image est grande, plus il y a de pixels à parcourir, ce qui augmente la complexité de l'algorithme et rend le traitement plus long et plus consommateur en termes de ressources.

Il peut également être plus difficile de détecter les contours sur des images plus grandes car il y a plus de bruit et de détails qui peuvent masquer les contours réels.

De plus, lorsque nous avons voulu réaliser l'algorithme de Sobel sur une image ayant comme dimensions 10 200 pixels de largeur et 5 734 pixels de hauteur (soit une résolution de 58 486 800), le FPGA n'a pas pu le faire car la résolution de l'image était telle qu'il ne possédait pas les ressources mémoires pour la traiter comme le montre l'image ci-dessous :

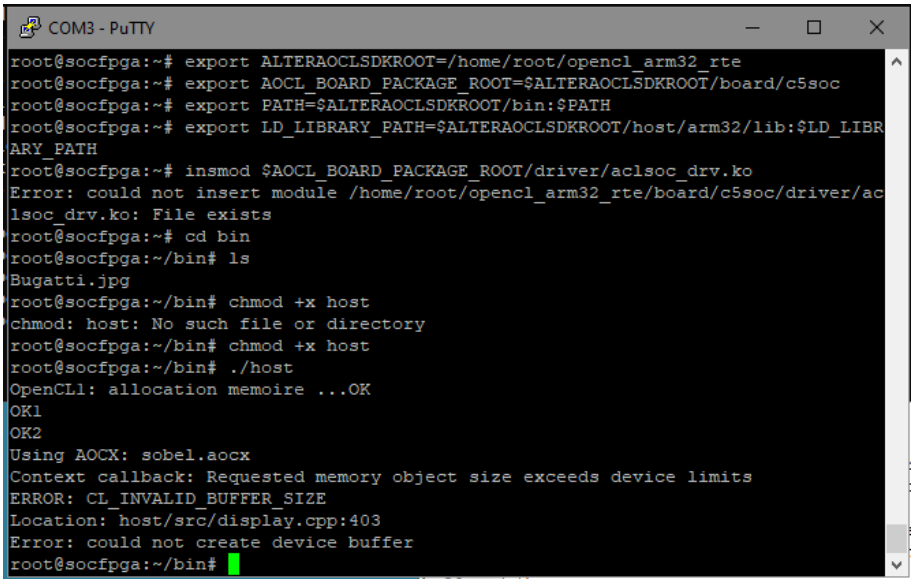



*Figure 11 : Limite de résolution que peut traiter le FPGA*

# 4. Évaluation des performances avec un autre algorithme de complexité différente

## 4.1 Explication et choix de l'algorithme

Nous avons voulu réaliser un algorithme de flou_gaussien. Un flou gaussien est un filtre de lissage utilisé pour enlever le bruit ou lisser les contours d'une image. Il fonctionne en utilisant un noyau de convolution, également appelé "kernel", qui est appliqué à chaque pixel de l'image d'entrée pour produire une image de sortie lissée.

Le kernel utilisé pour le flou gaussien est généralement une matrice carrée remplie de valeurs décrivant une distribution gaussienne. Cette distribution est centrée sur le pixel en cours de traitement et a une largeur définie par le paramètre sigma. Les valeurs du kernel sont normalisées pour s'assurer que la somme des valeurs est égale à 1.

En utilisant cette matrice, on peut calculer la valeur de chaque pixel de sortie en effectuant un produit de convolution entre le kernel et les pixels de l'image d'entrée dans une zone définie autour du pixel en cours de traitement. Les résultats de la convolution pour chaque pixel sont stockés dans l'image de sortie.

Le flou gaussien produit un effet de lissage sur l'image en atténuant les variations de luminosité locales, c'est-à-dire en enlevant les détails qui ont une amplitude de luminosité inférieure à celle du kernel. Il est particulièrement utile pour enlever le bruit d'une image sans altérer significativement les contours.

## 4.2 Implémentation de l'algorithme (langage OpenCL)

```
#include "../host/inc/defines.h"

__kernel
void flou_gaussien(global unsigned int * restrict entree_image, global unsigned int
* restrict sortie_image,
           const int iterations, const int sigma)
{
    int taille_kernel = (6 * sigma + 1);
    taille_kernel = taille_kernel < 3 ? 3 : taille_kernel; // la taille du kernel
est au moins de 3
    int centre_kernel = taille_kernel / 2;

    // Remplir le kernel avec des valeurs gaussiennes
    int kernel[taille_kernel];
    for (int i = 0; i < taille_kernel; i++) {
        int x = i - centre_kernel;
        kernel[i] = (int)(exp(-0.5 * x * x / sigma / sigma));
    }

    // Normaliser le kernel
    int somme = 0;
    for (int i = 0; i < taille_kernel; i++) {
        somme += kernel[i];
    }
    for (int i = 0; i < taille_kernel; i++) {
        kernel[i] = (int)(((float)kernel[i]/(float)somme)*100);
    }

    // Créer le tampon de pixels
    int taille_tampon = taille_kernel + iterations;
    int tampon[taille_tampon];

    // Initialiser le tampon avec des zéros
    for (int i = 0; i < taille_tampon; i++) {
        tampon[i] = 0;
    }

    // Convoluer l'image avec le kernel
    #pragma unroll iterations
    for (int i = 0; i < iterations; i++) {
        // Décaler le tampon
        #pragma unroll taille_tampon
        for (int j = taille_tampon - 1; j > 0; j--) {
            tampon[j] = tampon[j - 1];
        }
        tampon[0] = entree_image[i];

        // Convoluer le kernel avec le tampon
        int resultat = 0;
        #pragma unroll taille_kernel
        for (int k = 0; k < taille_kernel; k++) {
            resultat += tampon[k] * kernel[k];
        }
        sortie_image[i] = (unsigned int)(resultat/100);
    }
}
```

*Figure 12 : Code OpenCL d'un flou Gaussien*

Ce code est une implémentation d'un filtre de flou gaussien pour une image. Les étapes principales que nous avons dû suivre afin de réaliser l'algorithme sont les suivantes :
- Création d'un kernel Gaussien : Le kernel est une matrice de taille (6*sigma + 1) x (6*sigma + 1) qui est remplie avec des valeurs gaussiennes. La variable sigma est utilisée pour contrôler la largeur de la distribution gaussienne utilisée pour remplir le kernel
- Normalisation du Kernel : Les valeurs du kernel sont normalisées pour s'assurer que la somme des valeurs est égale à 1. Cela est fait pour s'assurer que l'intensité de l'image de sortie reste constante
- Création d'un tampon de pixels : un tampon est créé pour stocker les pixels de l'image d'entrée qui ont été décalés pour être utilisés lors de l'application du kernel
- Convolution de l'image avec le Kernel: Le Kernel est appliqué à chaque pixel de l'image d'entrée en utilisant une convolution. Cela consiste à décaler le kernel sur l'image d'entrée en utilisant un décalage de 1 pixel à chaque fois et à calculer le produit de chaque pixel
- Stockage des résultats : Les résultats de la convolution pour chaque pixel sont stockés dans l'image de sortie

Ainsi, ce code prend une image d'entrée, une image de sortie, le nombre d'itération pour l'application de l'effet de flou et un paramètre sigma qui va permettre de définir la largeur de la distribution gaussienne utilisée pour remplir le kernel. Il va ensuite effectuer une convolution de l'image d'entrée avec le kernel préalablement créé pour produire une image de sortie lissée.

## 4.3 Implémentation de l'algorithme (langage C++)

Voici ce que nous obtenons si nous reprenons notre code précédent en langage OpenCL et que nous le rendons maintenant séquentiel :

```
void flou_gaussien(global unsigned int * restrict entree_image, global unsigned int
* restrict sortie_image,
           const int iterations, const int sigma)
{
    int taille_kernel = (6 * sigma + 1);
    taille_kernel = taille_kernel < 3 ? 3 : taille_kernel; // la taille du
kernel est au moins de 3
    int centre_kernel = taille_kernel / 2;

    // Remplir le kernel avec des valeurs gaussiennes
    int kernel[taille_kernel];
    for (int i = 0; i < taille_kernel; i++) {
        int x = i - centre_kernel;
        kernel[i] = (int)(exp(-0.5 * x * x / sigma / sigma));
    }

    // Normaliser le kernel
    int somme = 0;
    for (int i = 0; i < taille_kernel; i++) {
        somme += kernel[i];
    }
    for (int i = 0; i < taille_kernel; i++) {
        kernel[i] = (int)(((float)kernel[i]/(float)somme)*100);
    }

    // Créer le tampon de pixels
    int taille_tampon = taille_kernel + iterations;
    int tampon[taille_tampon];

    // Initialiser le tampon avec des zéros
    for (int i = 0; i < taille_tampon; i++) {
        tampon[i] = 0;
    }

    // Convoluer l'image avec le kernel
    for (int i = 0; i < iterations; i++) {
        // Décaler le tampon
        for (int j = taille_tampon - 1; j > 0; j--) {
            tampon[j] = tampon[j - 1];
        }
        tampon[0] = entree_image[i];

        // Convoluer le kernel avec le tampon
        int resultat = 0;
        for (int k = 0; k < taille_kernel; k++) {
            resultat += tampon[k] * kernel[k];
        }
        sortie_image[i] = (unsigned int)(resultat/100);
    }
}
```

*Figure 13 : Code OpenCL d'un flou Gaussien*

## 4.4 Différence de complexité des algorithmes

### 4.4.1 Algorithme de Sobel

La complexité de l'algorithme de Sobel est de O(n) avec n étant le nombre d'itérations, cela est dû à la boucle principale « **while (count != iterations)** » qui est exécutée n fois. Il y a aussi une boucle imbriquée « **for** » qui est exécutée 3 fois et une autre boucle imbriquée « **for** » qui est exécutée 9 fois. Ces deux boucles ont une complexité de O(1) car elles sont exécutées un nombre fixe de fois. En résumé, la complexité globale de ce code est O(n).

### 4.4.2 Algorithme de flou Gaussien

La complexité de l'algorithme de flou Gaussien est de O(iterations * taille_kernel) car c'est la complexité la plus élevée (avec le plus grand nombre de boucle for imbriquée). Voici en détails les différentes boucles dans notre algorithme :
- La première boucle « **for (int i = 0; i < taille_kernel; i++)** » qui remplit le kernel avec des valeurs gaussiennes a une complexité de O(taille_kernel) car elle parcourt tous les éléments du kernel une seule fois
- La deuxième boucle « **for (int i = 0; i < taille_kernel; i++)** » qui normalise le kernel a une complexité de O(taille_kernel) qui est similaire à la première boucle
- La troisième boucle « **for (int i = 0; i < taille_tampon; i++)** » qui initialise le tampon avec des zéros a une complexité de O(taille_tampon).

- La boucle principale « **for (int i = 0; i < iterations; i++)** » qui convole l'image avec le kernel a une complexité de O(iterations) car elle parcourt toutes les itérations de l'image.
- La boucle interne **« for (int j = taille_tampon - 1; j > 0; j--)** » qui décale le tampon a une complexité de O(taille_tampon) qui est similaire à la boucle qui initialise le tampon.
- La dernière boucle interne « **for (int k = 0; k < taille_kernel; k++)** » qui convole le kernel avec le tampon a une complexité de O(taille_kernel) qui est similaire à la première boucle qui remplit le kernel.

En résumé, la complexité totale est de O(iterations * taille_kernel) car elle est déterminée par la boucle principale qui parcourt toutes les itérations de l'image et la boucle interne qui parcourt tous les éléments du kernel.